GUI Debugger

Dokumentace maturitní práce

Adam Zbořil

školní rok: 2025/2026 třída: 8.V

Zadání maturitní práce:

Cílem práce je vytvořit debugger s grafickým uživatelským prostředím v programovacím jazyce Rust. Hlavní funkcí programu je ladění kódu a programů kompilovaných programovacích jazyků. Program je pro použití pouze na operačním systému Linux a používá zejména knihovny iced (pro GUI) a nix, respektive GNU C Library.

Teoretická část – Kompilátory a interprety

Prohlašuji, že jsem na práci pracoval samostatně pouze za pomoci použitých zdrojů a že v práci i v dokumentaci jasně vymezuji, které části kódů jsou mým originálním dílem, které jsou upravenou verzí a které jsou převzaty v plném rozsahu.

---

podpis žáka

OBSAH

1. TEORETICKÁ ČÁST ... 6
1.1. KOMPILÁTORY ... 6
1.2. INTERPRETY ... 6
1.3. SILNÉ A SLABÉ STRÁNKY V POROVNÁNÍ ... 7
1.4. PRAKTICKÁ VYUŽITÍ ... 8
2. CÍLE PRÁCE ... 8
3. ZPŮSOBY ŘEŠENÍ A POUŽITÉ POSTUPY ... 9
3.1. NÁVRH A OBECNÁ ARCHITEKTURA ... 9
3.1.1. PRŮBĚH PROGRAMU ... 9
3.2. SLEDOVÁNÍ PROCESU ... 10
3.2.1. SPOUŠTĚNÍ PROGRAMU ... 10
3.2.2. OVLÁDÁNÍ PRŮBĚHU ... 10
3.2.3. ASYNCHRONNÍ SMYČKY ... 11
3.3. ZPRACOVÁNÍ INFORMACÍ K LADĚNÍ ... 11
3.3.1. ŘÁDKOVÝ PROGRAM A ZDROJOVÝ KÓD ... 11
3.3.2. INFORMACE O FUNKCÍCH ... 13
3.3.3. CALL STACK ... 14
3.3.4. ROZBALOVÁNÍ TYPŮ A ZOBRAZOVÁNÍ PROMĚNNÝCH ... 14
3.4. UŽIVATELSKÉ ROZHRANÍ ... 16
3.4.1. MŘÍŽKA PANELŮ ... 16
3.4.2. PANELY A DATA ... 17
3.5. CIZÍ KÓD ... 18
4. PROGRAMOVACÍ PROSTŘEDKY ... 19
4.1. PROGRAMOVACÍ JAZYK RUST, RUSTC A RUST-ANALYZER ... 19
4.2. VISUAL STUDIO CODE (VSC) ... 19
4.3. GNU MAKE ... 19
4.4. MICROSOFT COPILOT ... 19
4.5. POUŽITÉ KNIHOVNY ... 20
4.5.1. NIX ... 20
4.5.2. ICED ... 20
4.5.3. GIMLI ... 20
4.5.4. DALŠÍ ... 20
4.6. IKONY ADWAITA ... 21
5. ZHODNOCENÍ DOSAŽENÝCH VÝSLEDKŮ ... 21
6. INSTALACE ... 22

6.1. NÁROKY A KOMPATIBILITA ... 22
6.2. EXTERNÍ ZÁVISLOSTI ... 22
7. OVLÁDÁNÍ ... 23
7.1. POPIS UŽIVATELSKÉHO ROZHRANÍ ... 23
7.1.1. OKNO, TOOLBAR, PANELY ... 23
7.1.2. PANELY PODROBNĚ ... 24
7.1.3. DIALOGOVÁ OKNA ... 25
7.1.4. NASTAVENÍ ... 26
7.2. TYPICKÝ POSTUP PŘI POUŽÍVÁNÍ ... 26
7.2.1. PO SPUŠTĚNÍ ... 27
7.2.2. VYBRÁNÍ PROGRAMU A PŘÍPRAVA NA LADĚNÍ ... 27
7.2.3. ZAČÁTEK A PRŮBĚH LADĚNÍ ... 27
7.2.4. UKONČENÍ PROGRAMU ... 28
7.3. POZNÁMKY ... 28
8. SEZNAM POUŽITÝCH INFORMAČNÍCH ZDROJŮ ... 28

1. TEORETICKÁ ČÁST

Kompilátory a interprety tvoří základní kámen pro moderní programování. Oba typy softwaru umožňují spuštění abstraktního kódu, který se řídí pravidly daného programovacího jazyka, a tak zjednodušují celý proces programování.

1.1. Kompilátory

Kompilátory jsou programy, které překládají *zdrojový kód* do *cílového kódu*. Výstupní jazyk bývá zpravidla nižší programovací úrovně nežli ten vstupní.[1] Může to být například *strojový kód*, jazyk *Assembly* nebo *LLVM IR*.

Prvně probíhá tzv. *parsing* neboli čtení kódu uživatele a jeho syntaktická analýza; sestavují se funkce, proměnné, konstanty apod. Poté dochází k samotnému překládání: každý výraz ve vstupním jazyce má svůj definovaný zápis v jazyce výstupním. Takto se vytvoří hrubý náčrt výsledného kódu. Nakonec se podle logických a optimalizačních pravidel vytvoří výsledné uspořádání (případně se vynechají nepotřebné části), dosadí se číselné definice za pojmenovaní (anglicky „labels“) a výsledek je vypsán do souboru nebo standardního výstupu.

Nejzákladnějším příkladem kompilátorů je *assembler*, kterému stačí překládat uživatelsky čitelné instrukce do *strojového kódu* a dosazovat čísla. Pokročilejší kompilátory vyšších jazyků však umí mnohem více: makra, direktivy (příkazy), obecně definované struktury (tzv. *generics*) a jiné. Například *rustc* analyzuje validitu obsahu proměnných a upozorňuje na nestabilní funkce kódu.

Kompilace přímo do *strojového kódu* vyžaduje znalost architektury cílového procesoru, a proto se dnes už příliš nevyskytuje. Místo toho se používají podpůrné nástroje, například *LLVM*, které využívají obecný zápis k vytvoření samotného spustitelného souboru. Nové kompilátory tak nemusí znovu implementovat různé architektury. Často se průběh kompilace dá pozměnit: specifikace úrovně optimalizace, vytvoření informací k ladění kódu a další.

1.2. Interprety

Interprety umí spustit *zdrojový kód* i bez předchozí kompilace.[2] Podobně jako u kompilátorů se v první fázi kód *parsuje*, avšak ne jako celý soubor, nýbrž po částech (většinou po řádcích). Každá část představuje nějaký příkaz a ten se musí interpretovat (od toho také pochází název "interpret").

Interpretace můžeme dosáhnout několika způsoby. Nejjednodušší z nich je volání funkcí na základě shody textu. Tato metoda je rychlá a využívaná skriptovacími programovacími

jazyky (např. *BASH*).[3] Má však nevýhodu velikosti a paměťové zátěže samotného interpreta, protože všechny funkce musí být předem zkompilované.

Dalším způsobem je kompilace příkazu do pseudokódu; většinou se jedná o tzv. *bytecode*. Pseudokód je pak spuštěn pomocí obdobných pravidel jako v předchozí metodě, používá však *virtuální stroj*, který kód vyhodnocuje jako instrukce: výrazně rychleji a s menší zátěží na paměť.[2] Tato implementace je nejpoužívanější mezi interprety a najdeme ji například u jazyků *lua* a *python*.[4]

Jako nejrychlejší řešení bývá považována *JIT kompilace*, neboli překládání do *strojového kódu* za běhu. Po *parsování* je kód rovnou nebo s mezikrokem zkompilován a spuštěn přímo na úrovni procesoru. Využívá tak vysoké rychlosti, kterou mají kompilované jazyky, bývá však náročná na implementaci.[5]

1.3. Silné a slabé stránky v porovnání

Zásadní výhody plynoucí z kompilace jsou rychlost a efektivita. Kompilátor využívá více času na zpracování a vyhodnocení *zdrojového kódu*, a tak může odstranit nepoužité části programu a optimalizovat celkový průběh výsledného kódu. Taky umí vyprodukovat samostatný spustitelný soubor, který nevyžaduje externí software na spuštění. Navíc díky kompilaci můžeme spojit kód několika jazyků do jednoho programu.

Na druhou stranu kompilátory většinou potřebují přesné definice v kódu. Jsou tedy méně dynamické a jejich výsledná optimalizace záleží z velké části na *zdrojovém kódu*. Kompatibilita výsledného programu mezi systémy a architekturami je také velmi omezená a zpravidla vyžaduje jeho *rekompilaci*, nebo dokonce přepsání *zdrojového kódu*.

Interprety jsou používány zejména pro svou jednoduchost, volnost a interaktivnost. Kód interpretovaných jazyků bývá přehlednější a jednodušší na tvorbu, například díky dynamickému typování proměnných. Také dovolují spuštění kódu, který má v sobě logické či syntaktické chyby, což může, ale nemusí být vždy výhodou. Mimo jiné vytváří kompatibilní prostředí: pokud je interpret implementovaný pro danou architekturu nebo operační systém, můžeme spustit náš *zdrojový kód* i zde. Interprety navíc vybírají optimální funkce a optimalizace tedy nespoléhá pouze na programátorovi. Avšak neumí přeskočit nepotřebné části kódu.

Hlavní nevýhoda interpretace je již zmiňovaná časová, popřípadě paměťová náročnost. Kvůli *parsování* trvá spuštění déle a jazyky si většinou nemohou dovolit jinou metodu čištění paměti nežli tzv. „garbage collector“ (průběžné čištění nepotřebné paměti). Pro spuštění kódu

je také pokaždé potřeba přítomnost interpreta a nelze ho spustit v prostředí, které není daným interpretem podporované.

1.4. Praktická využití

Oba typy softwaru jsou kromě klasického programování využívány pro různé účely.

Další typy kompilátorů jsou například:[1]

- *cross-compilers* – kompilují kód do jiných architektur, než ve které jsou spuštěni
- *dekompilátory* – překládají *strojový kód* do programovacích jazyků vyšší úrovně
- *source-to-source kompilátory* – překládají kód z jednoho jazyka do druhého podobné úrovně
- *bootstrap kompilátory* – dočasné kompilátory pro kompilaci stabilnějších a lepších kompilátorů

Interprety dále umožňují například:[2]

- *virtualizaci* – použití jako virtuální stroj, spouštějící *strojový kód* jiné architektury
- *emulaci* – virtualizaci pro operační systémy nebo také simulaci architektur
- funkci jako *příkazové řádky* – využívající *REPL* (z anglického „read-eval-print loop“) pro interaktivní ovládání v textovém uživatelském prostředí

Prvními kompilátory byly *assemblery* a pochází z 40. let minulého století. Ty však popisovaly pouze ruční metodu pro překládání psaných instrukcí do *strojového kódu*. Naopak první interprety byly používány příkazovými jazyky vysoké úrovně k ovládání počítačů a procesorů. Velmi známým příkladem je *Microsoft BASIC*, který vznikl v roce 1975.[6]

2. CÍLE PRÁCE

Cílem mé práce bylo vytvořit debugger s grafickým uživatelským rozhraním. Program umí ladit kód kompilovaných jazyků, a tak pomáhá uživateli při hledání chyb v kódu. Nejdůležitější funkce ladění pro mě byly: možnost pozastavení programu, získání aktuální pozice v kódu, zobrazení stavu, ve kterém se nachází, čtení jeho paměti a zajištění komunikace skrze standardní vstup a výstup.

Má snaha byla hlavně vytvořit software, který je příjemný pro používání, dostatečně přizpůsobitelný a v ne menší řadě dále rozvíjitelný. Mimo jiné jsem chtěl získat další zkušenosti v oblasti operačního systému Linux, nízko-úrovňového programovaní a programovacího jazyka Rust.

3. ZPŮSOBY ŘEŠENÍ A POUŽITÉ POSTUPY

3.1. Návrh a obecná architektura

Program je navržený jako aplikace s grafickým prostředím. To znamená, že nemá daný postup spuštění, ale reaguje na vstup a příkazy uživatele. Jednotlivé moduly tedy spíše plní funkci knihoven nežli prosté rozdělení kódu do několika souborů.

Jádrem aplikace je proto uživatelské rozhraní. To používá strukturu **App**, ve které ukládá svůj stav a všechny informace potřebné k zobrazování grafiky. Pro zbytek dat, které nesouvisí s funkcí aplikace, jsem použil globální proměnné. O ně se stará soubor "data.rs". Tento přístup jsem zvolil kvůli optimalizaci a přehlednosti. Funkce používají méně parametrů a získal jsem možnost se vyhnout kopírování dat kvůli správě referencí.

Modul "dwarf.rs" je souborem funkcí, které zpracovávají informace k ladění. Jedná se o výpočetní komplex a mohl by být použit samostatně nebo v podobných projektech. Modul je zásadní pro ladění kódu na vyšší programovací úrovni.

Soubor "trace.rs" slouží k ovládání celého programu. Obsahuje funkce, které najdeme v naprosté většině debuggerů. Také funguje jako most při ladění mezi grafikou a procesy chystající data v pozadí. Soubor "object.rs" je jeho menším doplňkem, který se primárně stará o komunikaci se systémem.

Nakonec je celý program navržen, aby fungoval jako samostatný spustitelný soubor. Struktura **Asset** obsahuje všechny externí soubory, které jsou zkompilované přímo do kódu.

3.1.1. Průběh programu

Při spuštění programu se nejprve načte konfigurace. Přečte se nastavení uživatele a doplní se o výchozí hodnoty, popřípadě se použije výchozí nastavení jako takové. Poté se spustí samotná aplikace a otevře se hlavní okno.

Po vybrání souboru dochází k jeho zpracování. Nejdříve se načtou všechny sekce s informacemi pro ladění a zavolají se funkce **load_source** a **parse_functions**, které vytvoří data o *zdrojovém kódu* a funkcích. Toto přichystání dat snižuje výkon potřebný při samotném ladění.

Při zahájení ladění se otevře *pseudoterminál* a spustí se laděný program. Přečtou se také informace o tzv. *paměťovém rozložení* (anglicky "memory maps") a otevře se paměťový soubor procesu. V průběhu se vyhodnocují operace (pod typem **Operation**) a signály, které proces obdrží.

Vnitřní data se mění pouze při pozastavení, kdy se přepočítává většina informací o aktuálním stavu. Grafika tak při běžném používání nevyžaduje moc výkonu. Pokud je proces zastaven, popřípadě změněn vybraný soubor, tak se uvolňuje paměť od dat, která již nejsou potřebná.

3.2. Sledování procesu

Pro sledování se používá systémové volání *ptrace* [7] (anglicky "process trace"). Soubor "trace.rs" obsahuje funkce, které obalují toto systémové volání z knihovny *nix*. Při selhání některé z nich se také objeví dialogové okno s chybou.

3.2.1. Spouštění programu

K ladění programu potřebujeme, aby proces zavolal systémové volání *ptrace* s vlajkou "TRACEME". Tím prakticky říká nadřazenému programu, že může sledovat jeho průběh spuštění. Bez tohoto bychom potřebovali administrátorské pravomoci pokaždé, kdy aplikaci zapínáme.

Já ve svém kódu používám následující postup. Pomocí funkce *fork* nejdříve rozdělím proces na dva identické (ale se stejnou pamětí, takže nedochází ke zbytečnému kopírování dat). Poté se v podřadném z nich zavolá zmiňovaná vlajka a pak se spustí program, který chceme ladit. Také se propojí jeho standardní vstup a výstup s *pseudoterminálem*.

3.2.2. Ovládání průběhu

Proces vždy buď běží, nebo je pozastaven. Zastaví se, když narazí na *breakpoint* nebo obdrží signál. Signály jsou zpracovávány funkcí **handle**, která se stará o aktualizaci dat o stavu programu.

Breakpointy jsou instrukce v kódu o velikosti bajtu, které vyvolají přerušení programu. Operační systém poté zachytí toto přerušení a pošle nadřazenému procesu informaci o zastavení. Nahrazený bajt si musíme uschovat, abychom *breakpoint* také mohli odstranit.

Já ve svém kódu vytvářím typ **Breakpoints**, který představuje kolekci **HashMap**. Ta používá jako klíče *normalizované* adresy a jako hodnoty nahrazené bajty. *Normalizovaná* adresa neznačí přesnou pozici v kódu, nýbrž odchylku od začátku paměti programu. Při vkládání poté přičítám adresu, která značí začátek této paměti. Toto umožňuje zadávat *breakpointy* i před tím, než je program vůbec spuštěn.

Pokud je proces pozastaven, můžeme ho nechat pokračovat nebo spustit pouze nadcházející instrukci, udělat tzv. "krok". Při pokračování musíme vždy udělat "krok" před tím, než budeme zapínat *breakpointy* (jinak bychom naráželi pořád na ten samý).

"Krok ve *zdrojovém kódu*" používá jinou funkci nežli "krok". Na každou adresu, která je spojena s řádkem ve *zdrojovém kódu*, umístíme dočasný *breakpoint* a program necháme pokračovat. Toto děláme, protože nejsme schopni předvídat, jaký řádek je opravdu ten další (při volání funkce, smyčce a podobných případů).

3.2.3. Asynchronní smyčky

Pro čtení standardního výstupu z *pseudoterminálu* se používá asynchronní smyčka. Ta je vytvořena pomocí volání asynchronní funkce, která čeká na standardní výstup. Když dostane data, tak je vrátí a zavolá se operace **Operation::Read**. Výstup procesu se zpracuje a uloží.

Pokud funkce nevrátila chybu, tak se znovu stejná čtecí funkce zavolá a tím se vytvoří smyčka. Podobná smyčka je použita při čekání na signál. Ta se liší v tom, že sama sebe nevolá, avšak je zavolána při každém pokračování ve spuštění programu.

Tyto smyčky využívají rozhraní **Task**, které knihovna *iced* používá pro vykonávání asynchronních funkcí.

3.3. Zpracování informací k ladění

Většina kompilátorů v dnešní době používá standard *DWARF* [8] při vytváření informací pro ladění (tzv. "debug info"). Jedná se kolekci sekcí ve spustitelných souborech *ELF*; nejčastěji jsou označeny prefixem ".debug_". Program při načítání těchto sekcí nejprve testuje jejich přítomnost. Nejdůležitější informace jsou pro nás propojení *zdrojového kódu* s *kódem strojovým*, umístění funkcí ve *strojovém kódu* a získávání hodnot proměnných.

Program je rozdělen do několika *kompilačních jednotek*. Každá představuje nějaký samostatný celek kódu, například zkompilovaný soubor (nebo soubory) *zdrojového kódu* nebo modul. Obsahují definice funkcí, typů, proměnných; název, kompilační složku, programovací jazyk a veškeré informace potřebné ladění této jednotky.

3.3.1. Řádkový program a zdrojový kód

Každá *kompilační jednotka* má svůj tzv. *řádkový program* (anglicky "line program"). Ten obsahuje pseudokód, který specifikuje lokaci řádků *zdrojového kódu* ve *strojovém kódu*. Na získání dat z těchto programů používám struktury **SourceMap** a **LineAddresses** a funkci **load_source**.

```
The Directory Table (offset 0x22, lines 1, columns 1):
 Entry Name
 0     (indirect line string, offset: 0x7): /home/azi/code/3bd/examples/c

The File Name Table (offset 0x2c, lines 3, columns 2):
 Entry Dir     Name
 0     0       (indirect line string, offset: 0): main.c
 1     0       (indirect line string, offset: 0): main.c
 2     0       (indirect line string, offset: 0x25): other.h

Line Number Statements:
 [0x0000003b]  Set column to 12
 [0x0000003d]  Extended opcode 2: set Address to 0x1129
 [0x00000048]  Special opcode 8: advance Address by 0 to 0x1129 and Line by 3 to 4
 [0x00000049]  Set column to 9
 [0x0000004b]  Special opcode 118: advance Address by 8 to 0x1131 and Line by 1 to 5
 [0x0000004c]  Special opcode 104: advance Address by 7 to 0x1138 and Line by 1 to 6
 [0x0000004d]  Set column to 13
 [0x0000004f]  Special opcode 104: advance Address by 7 to 0x113f and Line by 1 to 7
 [0x00000050]  Set column to 10
 [0x00000052]  Advance PC by constant 17 to 0x1150
 [0x00000053]  Special opcode 21: advance Address by 1 to 0x1151 and Line by 2 to 9
 [0x00000054]  Advance PC by constant 17 to 0x1162
 [0x00000055]  Special opcode 48: advance Address by 3 to 0x1165 and Line by 1 to 10
 [0x00000056]  Set column to 12
 [0x00000058]  Special opcode 105: advance Address by 7 to 0x116c and Line by 2 to 12
 [0x00000059]  Set column to 1
 [0x0000005b]  Special opcode 76: advance Address by 5 to 0x1171 and Line by 1 to 13
 [0x0000005c]  Advance PC by 2 to 0x1173
 [0x0000005e]  Extended opcode 1: End of Sequence
```

Obrázek 1: Čitelný zápis řádkového programu s tabulkou souborů, které používá

LineAddresses je kolekce **HashMap**, která uschovává strukturu **SourceIndex** pod klíčem *normalizované* adresy. **SourceMap** také představuje **HashMap**, která uschovává vektor (standardní typ **Vec**) souborů *zdrojového kódu* pod klíčem kompilační složky.

Zmiňovaný **SourceIndex** je struktura obsahující řádek *zdrojového kódu*, kompilační složku a index ve vektoru souborů, který dostaneme, když dosadíme kompilační složku jako klíč do **SourceMap**. Jeden soubor *zdrojového kódu* ukládá struktura **SourceFile**. Ta obsahuje název souboru, respektive relativní cestu k souboru, odkaz na kompilační jednotku a pak obsah samotného souboru. Obsah se však nahrává až za chodu k ušetření paměti.

V nadhledu tedy **SourceMap** uschovává všechny soubory *zdrojového kódu* a **LineAddresses** ukládá páry adres a pozic ve *zdrojovém kódu*. Funkce **load_source** obě tyto struktury vytváří a pak ukládá do globálních proměnných. Informace získává skrze iteraci přes každou *kompilační jednotku* a poté vnitřní iteraci přes každou instrukci zmíněného pseudokódu. Ze všech instrukcí představující pozici v kódu se poté získá adresa, odpovídající řádek a soubor, ve kterém se nachází.

Ze souboru se dynamicky určí jeho kompilační složka. Pokud cesta k souboru používá relativní umístnění, přiřadí se výchozí kompilační složka celého programu. Pokud se jedná o absolutní cestu, tak algoritmus otestuje, jestli se část neshoduje s cestou výchozí jednotky. Pokud ano, tak se mu tato výchozí cesta přiřadí a přepočítá se relativní cesta k souboru. Jinak se použije absolutní cesta adresáře jako kompilační složka.

Tento proces zajišťuje větší přehlednost v ukládání *zdrojového kódu* a umožňuje jednoduší manipulaci s výchozí kompilační složkou. Poté se adresa a řádek uloží do **LineAddresses** a přidá se nový **SourceFile** do **SourceMap**, pokud se zde ještě nenachází.

Původně jsem chtěl použít místo **SourceIndex** pouze referenci na soubor v **SourceMap**. To však vyžadovalo tvorby *statické* reference, protože opouští funkci skrze globální proměnné. A při používání typu **Mutex** pro globální proměnné nelze *statickou* referenci vytvořit.

3.3.2. Informace o funkcích

Každá funkce má v sekci ".debug_info" záznam pod svou *kompilační jednotkou*. Záznam funkce obsahuje často název, lokaci ve *strojovém kódu*, typ návratové hodnoty a také pod sebou uschovává deklarace lokálních proměnných. Abychom jednodušeji hledali mezi těmito funkcemi, tak je předem zpracováváme do struktury **FunctionIndex**. O čtení těchto informací se stará funkce **parse_functions**.

```
<1><50>: Abbrev Number: 9 (DW_TAG_subprogram)
   <51>   DW_AT_external    : 1
   <51>   DW_AT_name        : (indirect string, offset: 0x5e): main
   <55>   DW_AT_decl_file   : 1
   <56>   DW_AT_decl_line   : 4
   <57>   DW_AT_decl_column : 5
   <58>   DW_AT_type        : <0x49>
   <5c>   DW_AT_low_pc      : 0x1129
   <64>   DW_AT_high_pc     : 0x4a
   <6c>   DW_AT_frame_base  : 1 byte block: 9c         (DW_OP_call_frame_cfa)
   <6e>   DW_AT_call_all_tail_calls: 1
   <6e>   DW_AT_sibling     : <0xb0>
```

Obrázek 2: Příklad záznamu funkce „main“

FunctionIndex slouží k zjištění aktuální funkce pomocí adresy a obsahuje tři kolekce **HashMap**. První schraňuje odkazy k funkcím pod klíči počáteční adresy funkce. Druhá ukládá intervaly všech funkcí v kompilační jednotce. Poslední obsahuje jména nadřazených celků ke každé funkci. Toto rozdělení do několika **HashMap** umožňuje rychlejší získávání dat při práci s většími projekty.

Většina záznamů má atributy "low_pc" a "high_pc", které udávají rozsah funkce. Zbylé používají sekci ".debug_ranges" – například pokud funkce obsahuje funkci vloženou. Podobně, každá deklarace bez rozsahu má zpravidla svou specifikaci, která obsahuje pro nás potřebný rozsah.

Funkce **parse_functions** tedy čte pouze rozsahy funkcí ve *strojovém kódu*. Iteruje přes všechny kompilační jednotky a v průběhu si ukládá v řadě názvy předchozích záznamů. Pokud záznam představuje deklaraci, neobsahuje rozsah funkce, je ale nutné si uložit název nadřazeného záznamu. Tento název je poté použit pro odlišení funkcí (například pokud ve dvou souborech najdeme funkci se stejný názvem).

3.3.3. Call stack

Při zastavení na řádce v kódu se zpravidla nacházíme v nějaké funkci. Neznáme však lokální proměnné a nevíme, jak jsme se na tuto pozici dostali. Proto rozbalujeme řadu volání funkcí neboli tzv. *call stack*.

Call stack získáme tak, že zjistíme, v jaké funkci se nacházíme, a vyhledáme si informace, které umožňují rozbalení stavu před zavoláním funkce, tzv. „unwind information“. Poté si přečteme pro nás zajímavé informace (lokální proměnné, parametry a další) a vyhodnotíme předchozí stav. Tento proces opakujeme, dokud se nedostaneme do hlavní funkce, která je označená symbolem "main" nebo specifikovaná v záznamech funkcí.

Hlavní částí zmíněných „*unwind information*“ je tzv. "cannonical frame address" (pod zkratkou CFA). *CFA* určuje pozici na *Stacku*, která představuje základní adresu pro aktuální funkci. Od ní se také odvíjí pravidla na získání předchozího stavu, které najdeme v sekci ".eh_frame". Díky *CFA* tak můžeme získat adresu, kam se funkce vrátí až dokončí svou operaci. Tuto adresu pak používáme pro rozbalení další funkce. *CFA* je také nezbytná pro čtení lokálních proměnných.

O rozbalení *call stacku* se stará funkce **unwind**, která do struktury **CallStack** ukládá vektor jednotlivých funkcí. Využívá také prakticky všechny přichystané informace k ladění. Nejdříve získá přes řádek (získaný pomocí adresy) *kompilační jednotku*. Ta nám společně s adresou umožňuje najít aktuální funkci. Poté se získají základní informace o funkci (jméno, typ návratové hodnoty, jméno nadřazeného záznamu) a vypočítá se hodnota *CFA*. Přes základní adresu se získají lokace parametrů a lokálních proměnných a načteme další informace o nich. Poté se rozbalí předchozí hodnoty registrů a získáme stav minulé funkce. Funkce **callstack** poté iteruje **unwind**, dokud se celý *call stack* nerozbalí.

3.3.4. Rozbalování typů a zobrazování proměnných

Při rozbalování se ukládají informace o lokálních proměnných a parametrech. Jedná se o název, typ, lokaci, popřípadě konstantní hodnotu. Zpracováváme však pouze proměnné, které byly již v kódu deklarované (pomocí porovnání aktuálního řádku s atributem v záznamu proměnné). Typ zatím ponecháváme pouze ve tvaru odkazu na záznam v sekci ".debug_info".

Hned po vytvoření struktury **CallStack** se zavolá metoda **stack_lines**, která ze zpracovaných informací vytvoří řádky textu, které již můžeme zobrazit. To zejména šetří výkon grafiky, avšak omezuje výstupní formát a původně jsem chtěl hodnoty vykreslovat přímo z jejich strukturní formy. Každý řádek obsahuje text a číslo hloubky, které používáme na vytvoření

odsazení řádků. Funkce **stack_lines** iteruje metodu **lines** struktury **Function**, ta zase metodu **lines** struktur **Variable** a **Parameter**.

Pro správné zobrazení hodnoty musíme znát typ proměnné nebo parametru. Na to používáme funkci **unwind_type**, která najde přes odkaz záznam typu a určí, o který se jedná (výčtový typ **TypeDisplay**). Všechny typy končí buď jako **BaseType**, **PointerType** nebo **EnumType**, jelikož ostatní typy vyžadují rozbalení typu, na který odkazují. Čtení z paměti získávající hodnoty proměnných se nachází pouze v metodách těchto typů.

Pro získání hodnot proměnných musíme tedy umět typy znovu rozbalit. Proto tato část kódu používá komplexní rekurzi na správné zobrazování hodnot. Prvně se rozbalí původní typ za tvorby typu **TypeDisplay**. Pak se na něm zavolá metoda **value**, která obecně volá metody **value** jednotlivých typů. Pokud se nejedná o zmíněné typy, tak se znovu zavolá funkce **unwind_type** tam, kde je potřeba a cyklus se opakuje.

BaseType používá atribut "encoding" na určení, jak danou hodnotu zobrazit. **PointerType** je vypsán vždy jako šestnáctkové číslo v ostrých závorkách. **EnumType** porovnává proměnnou s konstantními hodnotami *enumerátoru* a pak zobrazuje jeho název. **ModifierType** pouze přidá do textu své jméno a rozbalí další typ. **ArrayType** uloží pouze své jméno a jednotlivé elementy vypíše pod sebou. **StructType** funguje podobně jako **ArrayType**, ale nemá stanovenou délku, nýbrž jednotlivé členy s vlastním typem. **TypeDef** narozdíl od ostatních typů jméno nepřidává, ale odstraňuje celý rozbalený typ, podle kterého byl definován, a zanechá pouze poslední slovo (zpravidla hodnotu).

Pokud zobrazení jakéhokoliv typu selže, vypíše se na jeho místo '?'.

```
<2><35db>: Abbrev Number: 11 (DW_TAG_structure_type)
   <35dc>   DW_AT_name        : (indirect string, offset: 0x141e33): Message
   <35e0>   DW_AT_byte_size   : 40
   <35e1>   DW_AT_accessibility: 3     (private)
   <35e2>   DW_AT_alignment   : 8
<3><35e3>: Abbrev Number: 12 (DW_TAG_member)
   <35e4>   DW_AT_name        : (indirect string, offset: 0xc88a8): greeting
   <35e8>   DW_AT_type        : <0x34c7>
   <35ec>   DW_AT_alignment   : 1
   <35ed>   DW_AT_data_member_location: 32
   <35ee>   DW_AT_accessibility: 3     (private)
<3><35ef>: Abbrev Number: 12 (DW_TAG_member)
   <35f0>   DW_AT_name        : (indirect string, offset: 0x84464): text
   <35f4>   DW_AT_type        : <0x3694>
   <35f8>   DW_AT_alignment   : 8
   <35f9>   DW_AT_data_member_location: 0
   <35fa>   DW_AT_accessibility: 3     (private)
<3><35fb>: Abbrev Number: 12 (DW_TAG_member)
   <35fc>   DW_AT_name        : (indirect string, offset: 0xdfe34): length
   <3600>   DW_AT_type        : <0x39a7>
   <3604>   DW_AT_alignment   : 8
   <3605>   DW_AT_data_member_location: 24
   <3606>   DW_AT_accessibility: 3     (private)
```

Obrázek 3: Příklad záznamu struktury „Message“, která má 3 členy

3.4. Uživatelské rozhraní

Grafické rozhraní aplikace používá dvě hlavní funkce ze souboru "window.rs". Rozložení a vykreslování okna je dáno funkcí **view**, která volá funkce v souboru "ui.rs". Ty vždycky vyjímají ucelenou část grafiky pro větší přehlednost.

Druhá funkce **update** se naopak stará o akce a zajišťuje interaktivitu celého prostředí s uživatelem. K třídění akcí používá typ **Message**. Ten se dělí na **PaneMessage**, starající se o jednotlivé panely, **LayoutMesage**, který má na starost rozložení, a **Operation**, ovládající laděný proces.

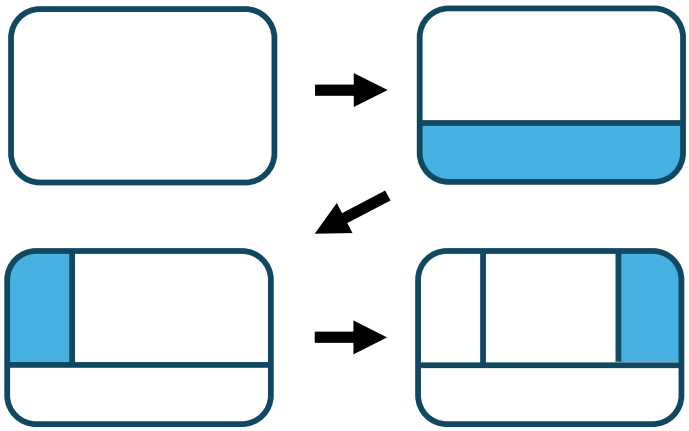
3.4.1. Mřížka panelů

Pro bohaté grafické přizpůsobení používám modul **pane_grid** z knihovny *iced*. Ten rozděluje plochu do několika menší panelů, které lze přesouvat a měnit jejich velikost. Tento návrh dále rozšiřuji o postranní panely, které se dají schovat. Informace o uspořádání vlastní struktura **Layout**.

Rozložení se řídí těmito pravidly: každá plocha je buď rozdělena na dvě další podle vodorovné či svislé osy, nebo obsahuje panel. Pomocí těchto jednoduchých pravidel vytvářím v metodě **base** základní rozvržení celé plochy. To se řídí tzv. módem spodního panelu a otevřenými či schovanými postranními panely.

Při schování postranního panelu se část plochy odebere a vloží do globální proměnné, abychom uložili vnitřní stav panelů. Proto se při změně postranních panelů zavolá funkce **layout**, která celé rozložení znovu vytvoří.

Každé rozdělení také obsahuje poměr, podle kterého je plocha rozdělena. Při změně velikosti se tato hodnota mění. Kvůli jednoduchým pravidlům se ale také mění velikost vnitřních ploch. Proto ve funkci **resize** používám specifický postup, který pomocí výpočtů s poměry zajišťuje, že velikost postranních panelů zůstává stejná.



Obrázek 4: Sestavování rozložení při módu plného spodního panelu

Na obrázku můžeme vidět příklad tvorby rozložení. Nejdříve se plocha rozdělí podle vodorovné osy za vzniku spodního panelu, poté podle svislé osy za vzniku levého panelu, a nakonec znovu podle svislé osy za vzniku středu a pravého postranního panelu. Poté se do středu a každého postranního panelu vloží jednotlivé menší panely.

3.4.2. Panely a data

Mřížka panelů je zobrazována pomocí funkce **mainframe**. Zde se také volá funkce **pane_view**, která přesměrovává vykreslování jednotlivých panelů. K manipulaci se stavem panelů používám funkci **pane_message**. Významné operace této funkce se týkají hlavně panelů **PaneCode** a **PaneMemory**.

Při vybrání souboru *zdrojového kódu* v **PaneCode** se asynchronně vytvoří seznam *breakpointů*. Původně jsem tuto operaci dělal při samotném vykreslování, ale vytvářela příliš velkou zátěž a prostředí se začalo sekat. Pokud vybraný soubor ještě nebyl načten, tak se ho program pokusí otevřít a přečíst.

PaneMemory používá velkou optimalizaci při načítání dat z paměti. Při každém čtení se totiž přečtou 4 KB a panel poté používá tato data a načítá nová pouze při opuštění stanoveného intervalu. Pokud jsme však blízko hranice přístupné paměti, čtení se nastaví na takovou hodnotu, abychom četli začátek nebo konec paměti. Posledním pravidlem je nečíst nová data, pokud poslední čtení používalo stejnou čtecí adresu (jelikož data již máme). Tento kód se nachází ve funkci **update_memory**.

```
pub fn update_memory(pane: &mut PaneMemory) { // Function Handling the bytes load to save performance
    let current: u64 = pane.address;
    let limit: u64 = pane.read_address;

    match test_memory(address: current) { // if we are outside of bounds, then we trigger read error
        Err(true) => {
            pane.read_error = true;
            return;
        },
        _ => ()
    };

    let mut new: u64 = if (current < limit + 4096 / 8) || (current > limit + 7*(4096 / 8)) { // if we are close the bounds of the LAST read, or outside of
        (current - current % 8) - 2048 // radius
    } else {
        return; // we are still well within the bounds so no need to reload the data
    };

    match test_memory(address: new) {
        Err(true) => new = get_map_range(address: current).unwrap().start, // if we are close to the beginning of the MAP, we set it to the start of the map
        _ => ()
    };

    match test_memory(address: new + 2048) { // if the new load would read past the current MAP, we set the beginning to be 4KB ahead of the end (reading al
        Err(false) => new = get_map_range(address: current).unwrap().end - 4096,
        _ => ()
    };

    if new == limit { // if last read address is the same as the new read, we already have the correct data
        return; // we have already done this
    };
```

Obrázek 5: Ukázka kódu funkce **update_memory**

Některá data panelů jsou z jejich podstaty uschována v celkovém stavu aplikace (například standardní výstup terminálu). Pro panely **PaneCode** a **PaneStack** se využívá vnitřní stav, ale jejich obsah může být aktualizován. To vidíme zejména při sjetí k řádku ve *zdrojovém kódu* při zastavení programu.

U **PaneStack** používáme unikátní číslo, které značí poslední aktualizaci panelu. To je důležité, protože panel může být skrytý a poté se neaktualizuje. Číslo je při každé změně inkrementováno a pokud se čísla stavu aplikace a panelu neshodují, tak se zobrazí tlačítko na nahrání aktuálních dat.

3.5. Cizí kód

Nakonec bych rád vymezil externí části kódu, které jsem ve svém projektu použil.

Při své práci jsem v souboru "trace.rs" vycházel z tutoriálu "Writing a Linux Debugger", který popisuje psaní debuggeru pro Linux v jazyce *c++*. Stejný postup jako zmíněný zdroj používám ve funkcích **insert_breakpoint** a **remove_breakpoint**, kde vyměňuji bajt v paměti laděného programu.[9] Dále také při operaci **Operation::Continue**, kde nejdřív udělám krok z aktuální pozice a pak až vkládám *breakpointy* a pokračuji ve spuštění programu [10], a ve funkci **handle**, ve které rozpoznávám naražení na *breakpoint*.[11]

V souboru "dwarf.rs" jsem se na několika místech inspiroval příklady z dokumentace knihovny *gimli*. Specificky ve funkci **load_source**, kde jsem použil příklad uvedený pro čtení řádkových programů.[12] Ve funkci **load_dwarf** používám kód na čtení sekcí *DWARF* pro vytvoření struktury **DwarfSections**.[13] A funkce **create_assembly** je napsána podle vzoru na stránkách dokumentace knihovny *iced-x86*.[14]

4. PROGRAMOVACÍ PROSTŘEDKY

4.1. Programovací jazyk Rust, rustc a rust-analyzer

Pro tento projekt jsem zvolil programovací jazyk Rust kvůli jeho stabilitě, bezpečnosti a rychlosti. Mám v něm také nejvíce zkušeností a chci se v něm dál rozvíjet. Pro samotné programování byly nejužitečnějšími nástroji kompilátor tohoto jazyka, zvaný *rustc*, a doplněk *rust-analyzer* pro *VSC*. Dohromady zajišťovaly příjemné vývojové prostředí: doplňování a navrhování funckí a metod a podrobné vysvětlení chyb v kódu.

4.2. Visual Studio Code (VSC)

Na psaní kódu jsem používal textový editor *VSC*. Pro účely programování mi vyhovuje z několika důvodů:

- široká nabídka doplňků a jejich jednoduchá správa
- bohaté zvýrazňování v kódu
- možnost kompletního uživatelského přizpůsobení
- rychlé hledání v textu a funkce multi-kursoru

4.3. GNU Make

V mém projektu se také objevuje několik souborů skriptovacího jazyka *GNU Make*. *GNU Make* zjednodušuje celý proces kompilace a používám ho jak pro samotný debugger, tak pro příklady ve složce "examples".

4.4. Microsoft Copilot

Při práci na projektu jsem také v některých částech použil kód, popřípadě konzultaci, od generativní umělé inteligence Copilot. Rád bych tyto pasáže zde vyjmenoval a upřesnil:

V souboru "data.rs" jsem si nevěděl rady s globálními proměnnými. Copilot mi doporučil použití typu **Mutex** ze standardní knihovny, který zajišťuje synchronizaci čtení a zápisu. Také jsem zjistil, jak spravovat statické reference, a to hlavně pro ukládání hrubého obsahu laděného programu.

V souboru "object.rs" jsem si nechal vysvětlit, jak fungují *pseudoterminály* a jak z nich vytvořit standardní komunikaci s laděným kódem.

V souboru "dwarf.rs" mi Copilot vygeneroval makro **impl_from_bytes**, které implementuje metody převádějící bajty do různých typů čísel, abych nemusel vypisovat repetitivní kód (specificky manuální implementaci pro každý typ). Také funkce **align_pointer** pro vytváření *asemblerového kódu* byla navržená Copilotem.

A nakonec jsem potřeboval vysvětlit proces a implementaci získávání návratové adresy funkcí (anglicky „return address“). Copilot mi nabízel vypočítávání pozice s touto adresou přes odchylku, já jsem však nakonec použil vnitřní funkci knihovny *gimli*, která odvozuje tyto hodnoty přes informace k ladění.

Všechny zmíněné části kódu jsou označené komentářem "//AI".

4.5. Použité knihovny

4.5.1. nix

Knihovna *nix* [15] je nadstavbou Standardní knihovny pro jazyk C (známá také jako "libc"). Použil jsem ji hlavně pro moduly *ptrace*, *process* a *term*. *Ptrace* umožňuje sledování procesů, z *process* jsem použil funkci *fork* pro spuštění nového podprocesu a *term* byl nezbytný pro vytvoření *pseudoterminálu* na standardní komunikaci.

4.5.2. iced

Knihovna *iced* [16] přináší prostředí pro jednoduchou tvorbu grafických aplikací. I přes to, že je stále ve vývoji, nabízí bohatou škálu přehledných funkcí pro tvorbu grafiky a interakce s uživatelem. Taky jsem s ní s měl trochu zkušeností.

4.5.3. gimli

Knihovna *gimli* [17] spravuje informace pro ladění, zejména ty pod standardem *DWARF*. Jedná se o velmi silný modul na *parsování* těchto dat. Používám ji zejména pro zobrazování *zdrojového kódu* a sledování lokálních proměnných.

4.5.4. Další

Mimo hlavní knihovny jsem dále použil:

- *iced-x86* [14] jako *disassembler strojového kódu* pro ladění i na nižší úrovni
- *toml* [18] pro nahrávání a *parsování* konfiguračních souborů

- *rfd* [19] na vybírání programu skrze průzkumníka souborů a také komunikaci s uživatelem přes tzv. dialogová okna
- *rust-embed* [20] pro kompilaci všech externích souborů (tzv. *assets*) do výsledného spustitelného kódu (vytvoření tzv. „single-file binary executable“)
- *std* [21] jako standardní knihovnu jazyka *Rust* pro obecné funkce, struktury a další

Široký seznam knihoven byl pro mě bližší nežli závislost projektu na externích programech.

4.6. Ikony Adwaita

V práci také používám soubor ikon *Adwaita* [22] vytvořený pod projektem *GNOME*. Ve složce "code/assets/icons" se s použitými ikonami nachází soubory "LICENSE" a "README.md", které upřesňují licencování této grafiky. Všechny ikony, vyjma souboru "TBD.svg", používám pod licencí "GNU Lesser General Public License v3.0". Ta je kompatibilní s tou, která licencuje celý můj projekt ("GNU General Public License v3.0").

5. ZHODNOCENÍ DOSAŽENÝCH VÝSLEDKŮ

Ačkoliv jsem nedosáhl všech mých původních představ, věřím, že jsem zadání splnil a osobně jsem s prací velmi spokojený. Myslím si, že jsem dokázal vytvořit software, který je funkční, přizpůsobitelný a rozvíjitelný. Rád bych však vyjmenoval některé nedostatky, které v práci cítím.

Chybí mi podrobnější barevné zvýrazňování v aplikaci, hlavně v panelech **PaneCode** a **PaneStack**. Stejně tak jsem zjistil při zkoušení dalších vestavěných barevných schémat z knihovny *iced*, že u některých z nich je text v programu špatně čitelný.

Dále mi nevyhovuje funkce panelu **PaneAssembly**, jelikož se nedá prohlížet *asemblerový kód* daleko od aktuální pozice v kódu. Určitě bych také přidal panelu **PaneStack** možnost zobrazení hodnot proměnných v různých formátech a také funkci připnutí některých řádků pro větší přehlednost.

A nakonec mi chybí klávesové zkratky ovládající aplikaci a vybírání souboru přes argumenty příkazu na spuštění aplikace. Program plánuji nadále vylepšovat a rozšiřovat, hlavně za účelem odstranění zmíněných nedostatků.

6. INSTALACE

Ve složce „build“ se nachází soubor „Makefile“, který obsahuje script pro program *Make*. Stačí tedy v tomto adresáři zadat příkaz "make" (nebo "make all") a celý program se zkompiluje a vytvoří se zde spustitelný soubor "tbd". Ten se dá spustit pomocí "./tbd". (Pro instalaci na použití kdekoliv v systému můžeme přesunout soubor do "/bin" nebo "/usr/bin" nebo přidat cestu k souboru do systémové proměnné "$PATH".)

Program můžeme spustit i mimo prostředí terminálu.

6.1. Nároky a kompatibilita

Program podporuje pouze operační systém Linux, měl by však fungovat na většině distribucí, v desktopových prostředích typu *x11* i *wayland*. Je funkční výhradně na procesorech architektury *AMD64* (*x86_64*).

Výsledný soubor má 21 MB a z mého testování program nikdy nevyžadoval více než 120 MB paměti. Celková zátěž paměti se však zvyšuje dvojnásobně s velikostí laděného kódu, protože jeho obsah je načten pro zpracování a pak ještě spuštěn. Debugger vyžaduje největší výkon při zvolení souboru na zpracování dat. Při velkém množství informací k ladění může tento proces trvat i několik vteřin, například u programů psaných v jazyce *Rust*. Ke zpomalení grafiky také může dojít, pokud debugger zobrazuje velký text *zdrojového kódu*.

Program využívá tzv. *procfs*.[23] Jedná se o správu běžících procesů skrze souborový systém. Pro správný chod programu je nutné, aby prostředí toto rozhraní podporovalo. O tom se můžeme ujistit promocí příkazu "test -d /proc && echo true" (příkaz vypíše "true", pokud najde složku tohoto rozhraní).

Všechen kód byl vyvíjen a důkladně testován v distribuci Debian 12 Bookworm a v prostředí *wayland*, specificky v KDE Plasma. Úplná kompatibility byla také zajištěna pro Debian 13 Trixie.

6.2. Externí závislosti

Na kompilaci kódu je potřeba sada nástrojů pro jazyk Rust s názvem *rustup*.[24] Manuální instalace použitých knihoven není potřebná, o ty se stará nástroj "cargo" z této sady nástrojů.

Dále pro dialogová okna je nutný program *zenity*.[25] Tato závislost není povinná, je ale silně doporučená.

Projekt používá *GNU Make* [26] na jednodušší kompilaci a jeho instalace je pro zajištění správného fungování programu potřebná.

Pro kompilaci příkladů kódu ve složce "examples" jsou vyžadovány kompilátory GNU C Compiler a GNU C++ Compiler [27], také známé jako *gcc* a *g++*.

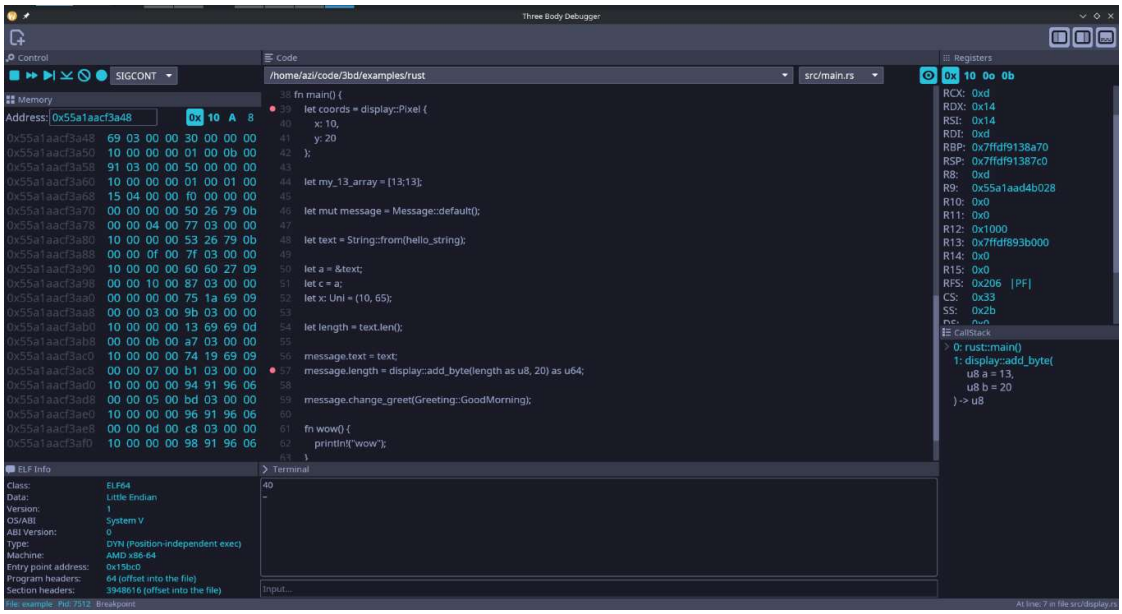
A v neposlední řadě je nutná Knihovna GNU C, také známa jako *glibc*. Na distribuci Debian se specificky jedná o balík *libc6*.

Popis instalace těchto závislostí pro distribuce Debian a Ubuntu se nachází v souboru "DEPENDENCIES.md".

7. OVLÁDÁNÍ

7.1. Popis uživatelského rozhraní

Celé uživatelské rozhraní se skládá ze tří částí. Tou nejdůležitější je hlavní okno, přes které se program ovládá, a kde se zobrazují všechny informace při ladění kódu. Další jsou menší dialogová okna, která sdělují uživateli zásadní zprávy v průběhu používání. Třetí částí je nastavení skrze konfigurační soubory.



Obrázek 6: Vzhled hlavního okna aplikace

7.1.1. Okno, toolbar, panely

Okno se dělí na *toolbar* nahoře, *statusbar* dole, a *mainframe* uprostřed. *Toolbar* obsahuje vlevo tlačítko na vybrání souboru a vpravo tlačítka na otevření či schování postranních panelů. *Statusbar* obsahuje obecné informace jako název vybraného souboru, identifikační číslo procesu (tzv. *PID*), stav procesu a popřípadě na jakém řádku *zdrojového kódu* je proces zastaven.

Mainframe obsahuje panely, které zobrazují veškeré informace o procesu a umožňují ladění kódu. Tyto panely můžeme přesouvat, měnit jejich velikost, a tak si přizpůsobit celé grafické prostředí. Každý panel má svůj titulek se jménem a pod ním svůj obsah. V horní části obsahu se soustřeďují interaktivní struktury, například tlačítka, výběry ze seznamů a další. Většina panelů má své vnitřní hodnoty, a proto můžeme používat i více panelů stejného typu zároveň.

7.1.2. Panely podrobně

Panelů je celkem 8 a každý plní jinou funkci.

Panel "Control" obsahuje tlačítka na ovládání laděného procesu. Zleva doprava jdou následovně: "Spustit/Zastavit", "Pokračovat/Pozastavit", "Krok", "Krok ve *zdrojovém kódu*", "Ukončit" a "Poslat Signál". Úplně vpravo poté můžeme vybrat signál, který chceme procesu poslat. "Krok" znamená spustit další instrukci, zatímco "Krok ve *zdrojovém kódu*" spustí program a zastaví ho, jakmile narazí na pozici, která je spojená s řádkem ve *zdrojovém kódu*.

Panel "Memory" zobrazuje obsah paměti programu. V poli můžeme zadat adresu v šestnáctkovém nebo desítkovém zápisu. Tlačítko úplně vpravo mění počet zobrazovaných bajtů na řádek. Zbylá tlačítka specifikují formát, ve kterém jsou jednotlivé bajty zobrazeny. V těle panelu se pak nachází řádky s adresami a bajty. Adresa vždy ukazuje na pozici prvního bajtu v řádku. Při použití kolečka na myši se můžeme pohybovat nahoru a dolu.

Panel "Code" zobrazuje zdrojový kód laděného programu. Nahoře můžeme vybrat tzv. kompilační složku a soubor. Tlačítko vpravo udává, jestli chceme panel aktualizovat, a tak sledovat aktuální pozici v kódu. Pozice, na které právě jsme, je zobrazena přes zvýrazněné číslo řádku. Pokud řádek má k sobě přiřazenou adresu, můžeme na něj dát *breakpoint*. Tlačítka na *breakpoint* se nachází vlevo od čísla řádků.

Panel "Registers" zobrazuje hodnoty registrů procesoru. Tlačítka nahoře mění číselnou soustavu, ve které jsou hodnoty zobrazovány.

Panel "ELF Info" vypisuje informace, které nalezneme v hlavičce souborů *ELF*. Jedná se například o cílový operační systém a architekturu, vstupní adresu kódu a další.

Panel "Terminal" funguje jako vnitřní terminál pro standardní komunikaci s laděným programem. Nahoře je okno se standardním výstupem, dole je pole pro standardní vstup. Aktuální pozice kursoru je zobrazována znakem '_'. Pro poslání standardního vstupu je nutné zmáčknout klávesu "Enter".

Panel "Assembly" vypisuje *asemblerový kód* okolo aktuální pozice. Ta je vyznačena zvýrazněnou adresou vlevo. Instrukce jsou zobrazovány ve formátu assembleru *NASM*. Jsou vypsány také bajty každé instrukce a vedle adres najdeme tlačítka na *breakpointy*.

Panel "CallStack" vypisuje tzv. *call stack* neboli seznam funkcí podle toho, jak byly postupně volány. U každé funkce pak vypisuje deklarované proměnné a jejich typy a hodnoty. Řádky značící zavolání funkce jsou zvýrazněny. "0:" znamená prvně zavolaná funkce, zpravidla "main". Pole funkcí, struktur a řetězců jsou zobrazovány s odsazením pro lepší čitelnost. Vedle řádků se proto nachází tlačítka na schování či rozbalení odsazených polí. Při aktualizaci je vždy obsah aktuální funkce rozbalen, zatímco všech ostatních schován.

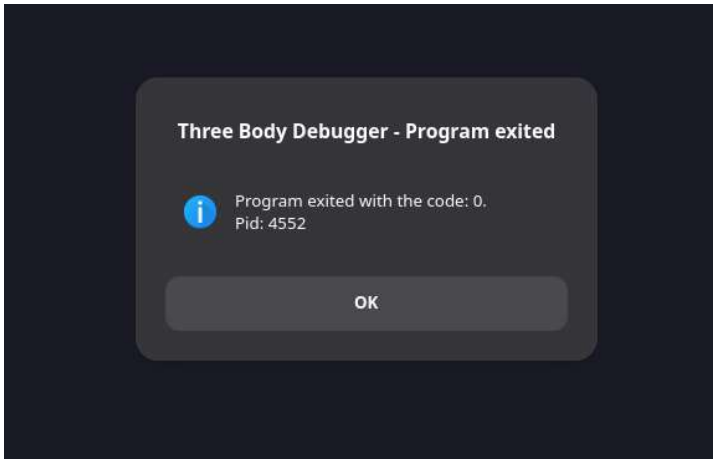
7.1.3. Dialogová okna

Dialogová okna sdělují důležité informace uživateli. Jedná se o chyby při ladění kódu, informace o průběhu spuštěného procesu a varování při nestandardních situacích.

Zprávy o chybách vždy zobrazí příčinu chyby a případně další informace o selhání některé z funkcí. Například při pokusu o čtení neexistující lokace v paměti vyskočí okno s touto informací a s kódem chyby. Při skončení programu nebo zastavení přes signál se zobrazí okno upozorňující uživatele o této skutečnosti.

Některé zprávy vyžadují rozhodnutí uživatele. Jeden z případů je selhání funkce, která čeká na zastavení sledovaného procesu. Uživatel má možnost tento pokus opakovat nebo proces zastavit. Druhý nastane, pokud uživatel chce vybrat nový soubor a starý program stále běží.

Dialogové zprávy zvyšují interaktivitu s uživatelem a zamezují přehlédnutí zásadních informací.



Obrázek 7: Dialogové okno upozorňující na skončení sledovaného programu

7.1.4. Nastavení

Uživatel může upřesnit nastavení aplikace v konfiguraci. Ta je psána v textové podobě ve formátu *TOML* [28] a nachází se v adresáři "~/.config/tbd/config.toml", kde '~' značí domovský adresář uživatele. Je nutné podotknout, že tento soubor ani adresář aplikace sama nevytváří. (Pro použití stačí vytvořit složku "tbd" v adresáři "~/.config/" a v ní pak vytvořit soubor "config.toml")

Nastavení se dělí na několik částí. Nastavení "window" upřesňuje pozici a velikost okna při spuštění a také barevné schéma aplikace. V sekci "layout" můžeme nastavit rozložení a otevřené postranní panely při spuštění. V "layout.panes" si můžeme vybrat, které panely chceme používat. Můžeme dokonce mít více panelů stejného typu.

```
37
38 [layout]
39 status_bar = true
40 sidebar_left = true
41 sidebar_right = true
42 panel = true
43 panel_mode = "left" # "middle", "left", "right", "full"
44
45 [layout.panes]
46 main = ["code"] # next to each other
47 left = ["control", "memory"] # under each other
48 right = ["registers", "stack"] # under each other
49 panel = ["info", "terminal"] # next to each other
50
51 # Available Panes:
52 # control
53 # code
54 # memory
55 # info
56 # registers
57 # terminal
58 # assembly
59 # stack
60
```

Obrázek 8: Ukázka ze souboru výchozí konfigurace ve formátu TOML

Poslední část se jménem "feature" slouží k používání experimentálních funkcí. Aktuálně se jedná pouze o funkci, která zajišťuje správné rozbalování řady *call stack* u programů, které byly psány v jazyce *Rust*.

Není nutné specifikovat všechna nastavení, zbytek se automaticky doplní výchozími hodnotami. Výchozí nastavení je k nalezení v souboru "code/assets/config.toml". Zde jsou také vypsané všechny možnosti a další popis jednotlivých položek.

7.2. Typický postup při používání

Programy, které chceme sledovat bychom měli zkompilovat tak, aby se v spustitelném souboru vytvořili informace pro ladění. Nejčastěji to znamená přidat vlajku "-g" při zadávání příkazu ke kompilaci. Tyto informace se vytvoří nehledě na přítomnosti jiných vlajek,

například optimalizačních. Pro nejlepší průběh ladění je však doporučeno nepřidávat jiné vlajky nežli pro ladění.

7.2.1. Po spuštění

Po otevření hlavního okna si můžeme přemístit panely tam, kde nám vyhovují a upravit si jejich velikost. Poté vybíráme soubor přes tlačítko na *toolbaru* vlevo.

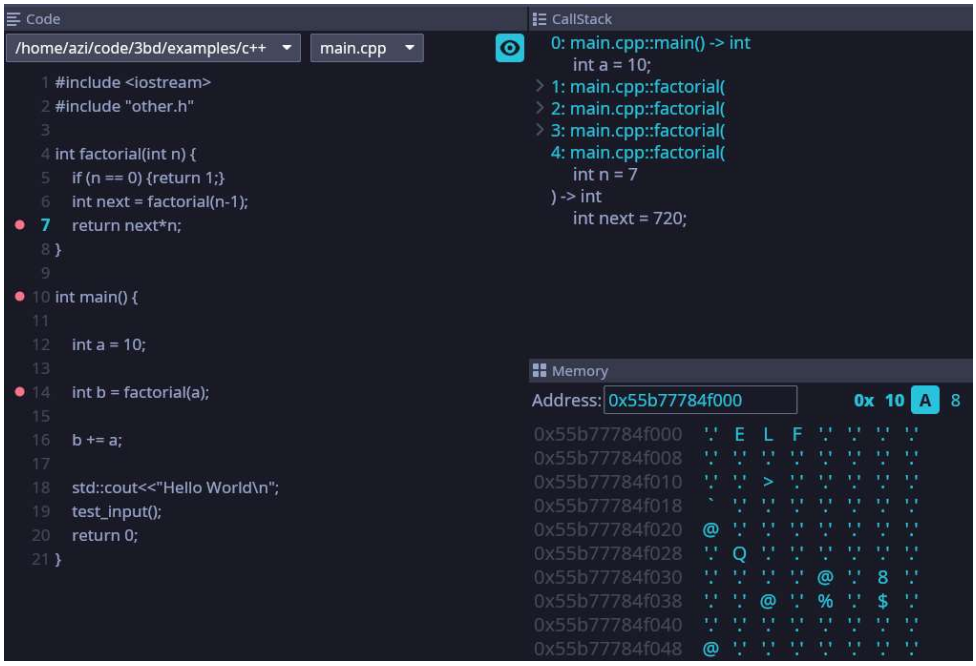
7.2.2. Vybrání programu a příprava na ladění

Vybereme spustitelný soubor a počkáme na jeho zpracování. Poté co se načte si můžeme prohlédnout *zdrojový kód* a umístit *breakpointy* na problematická místa. Je obecně dobré si umístit alespoň jeden někam na začátek kódu funkce "main". Poté můžeme spustit program přes tlačítko vlevo na panelu "Control".

7.2.3. Začátek a průběh ladění

Zobrazí se nám obsah panelů "Memory", "Registers", "Terminal" a "Assembly". Už teď si můžeme prohlížet informace o procesu. Program je po spuštění zastaven. Není doporučeno používat tlačítko pro "Krok ve *zdrojovém kódu*" dokud nevstoupíme do hlavní funkce, zpravidla "main".

Můžeme tedy kód nechat běžet až narazí na nějaký z *breakpointů*. Když se tak stane, můžeme si prohlédnout hodnoty lokálních proměnných, výstup z terminálu a přidat nebo odebrat některé *breakpointy*.



Obrázek 9: Ukázka z aplikace při zastavení na breakpointu

Breakpointy bychom nikdy neměli umisťovat, pokud kód právě běží. Na to máme možnost si kód pozastavit (2. tlačítko panelu "Control", když kód zrovna běží). Také bychom neměli pozastavovat kód, pokud program právě čeká na standardní vstup z terminálu. Takto pokračujeme dál, tak jak potřebujeme.

7.2.4. Ukončení programu

Pokud kód skončí, zjistíme to přes dialogové okno a dozvíme se hodnotu konečného kódu (anglicky "exit code"). Program můžeme ukončit i sami přes tlačítko "Zastavení" nebo "Ukončení" (ukončení posílá signál SIGKILL). Stejný program můžeme zkusit spustit znovu nebo klidně nahrát nový. Pokud byl program ukončen signálem, tak je nám sděleno, jaký signál to byl.

7.3. Poznámky

Laděný program by měl být vždy ukončen před zavřením samotné aplikace. Při ukončení debuggeru dojde totiž k odpojení podprocesu a ten pak pokračuje sám v normální chodu.

Aplikace se také může dostat do stavu, kdy neodpovídá. Naprosto nejpravděpodobnější příčina je, pokud je právě zobrazeno dialogové okno. Okno je potřeba tedy odkliknout a program se pak vrátí do chodu. Toto není chyba a jedná se o účelnou implementaci.

Debugger neumí ladit asynchronní funkce (vlákna) a podřadné programy, které vznikly za běhu laděného procesu. Správný průběh ladění jsem ani netestoval s programy, které tyto funkce používají. Ladění za těchto podmínek je tedy **silně nedoporučeno** kvůli nepředvídatelnosti průběhu laděného procesu.

8. SEZNAM POUŽITÝCH INFORMAČNÍCH ZDROJŮ

[1] *Compiler* [online]. Wikipedie: Otevřená encyklopedie, 2026-03-04. [cit. 2026-03-29]. Dostupné z: https://en.wikipedia.org/wiki/Compiler

[2] *Interpreter (computing)* [online]. Wikipedie: Otevřená encyklopedie, 2026-01-22. [cit. 2026-03-29]. Dostupné z: https://en.wikipedia.org/wiki/Interpreter_(computing)

[3] *Bash (Unix shell)* [online]. Wikipedie: Otevřená encyklopedie, 2026-03-26. [cit. 2026-03-29]. Dostupné z: https://en.wikipedia.org/wiki/Bash_(Unix_shell)

[4] *Python (programming language)* [online]. Wikipedie: Otevřená encyklopedie, 2026-03-25. [cit. 2026-03-29]. Dostupné z: https://en.wikipedia.org/wiki/Python_(programming_language)

[5] *Just-in-time compilation* [online]. Wikipedie: Otevřená encyklopedie, 2026-03-06. [cit. 2026-03-29]. Dostupné z: https://en.wikipedia.org/wiki/Just-in-time_compilation

[6] *Microsoft BASIC* [online]. Wikipedie: Otevřená encyklopedie, 2026-01-17. [cit. 2026-03-29]. Dostupné z: https://en.wikipedia.org/wiki/Microsoft_BASIC

[7] KERRISK Michael. *System Calls Manual: ptrace(2)* [online]. 2026-01-16. [cit. 2026-03-29]. Dostupné z: https://man7.org/linux/man-pages/man2/ptrace.2.html

[8] *DWARF Debugging Standard* [online]. DWARF Debugging Information Format Committee, 2025. [cit. 2026-03-29]. Dostupné z: https://dwarfstd.org/

[9] BRAND, Sy, 2017. *Writing a Linux Debugger Part 2: Breakpoints* [online]. 2025. [cit. 2026-03-29]. Dostupné z: https://tartanllama.xyz/posts/writing-a-linux-debugger/breakpoints/

[10] BRAND, Sy, 2017. *Writing a Linux Debugger Part 6: Source-Level Stepping* [online]. 2025. [cit. 2026-03-29]. Dostupné z: https://tartanllama.xyz/posts/writing-a-linux-debugger/source-level-stepping/

[11] BRAND, Sy, 2017. *Writing a Linux Debugger Part 5: Source and Signals* [online]. 2025. [cit. 2026-03-29]. Dostupné z: https://tartanllama.xyz/posts/writing-a-linux-debugger/source-and-signals/

[12] FITZGERALD Nick, CRAIG Philip, 2025. *A simple example of parsing `.debug_line`* [online]. 2025-12-12. [cit. 2026-03-29]. Dostupné z: https://github.com/gimli-rs/gimli/blob/main/crates/examples/src/bin/simple_line.rs

[13] FITZGERALD Nick, CRAIG Philip, 2025. *A simple example of parsing `.debug_info`* [online]. 2025-11-11. [cit. 2026-03-29]. Dostupné z: https://github.com/gimli-rs/gimli/blob/main/crates/examples/src/bin/simple.rs

[14] *iced_x86, v1.21.0* [online]. Rust Foundation, 2026. [cit. 2026-03-29] Dostupné z: https://crates.io/crates/iced-x86

[15] LERCHE Carl, 2026. *nix, v0.31.2* [online], 2026-02-28. [cit. 2026-03-29]. Dostupné z: https://crates.io/crates/nix

[16] HÉCTOR, 2026. *iced* [online]. 2026-03-27. [cit. 2026-03-29]. Dostupné z: https://iced.rs/

[17] FITZGERALD Nick, CRAIG Philip, 2026. *gimli, v0.33.1* [online]. 2026-03-29. [cit. 2026-03-29]. Dostupné z: https://crates.io/crates/gimli

[18] HUSS Eric, PAGE Ed, 2026. *toml, v1.1.0* [online]. 2026-03-23. [cit. 2026-03-29]. Dostupné z: https://crates.io/crates/toml

[19] MARYŃCZAK Bartłomiej, 2026. *rfd, v0.17.2* [online]. 2026-01-12. [cit. 2026-03-29]. Dostupné z: https://crates.io/crates/rfd

[20] JOHN Peter, 2026. *rust-embed, v8.11.0* [online]. 2026-01-14. [cit. 2026-03-29]. Dostupné z: https://crates.io/crates/rust-embed

[21] *The Rust Standard Library, v1.94.1* [online]. Rust Team, 2026-03-28. [cit. 2026-03-29]. Dostupné z: https://doc.rust-lang.org/std/

[22] *Adwaita Icon Theme* [online]. GNOME Project, 2026. [cit. 2026-03-29]. Dostupné z: https://github.com/GNOME/adwaita-icon-theme

[23] KERRISK Michael, 2026. *File Formats Manual, proc(5)* [online]. [cit. 2026-03-29]. Dostupné z: https://man7.org/linux/man-pages/man5/proc.5.html

[24] *Rustup: the Rust toolchain installer* [online]. Rust Team, 2026. [cit. 2026-03-29]. Dostupné z: https://rustup.rs/

[25] *zenity* [online]. GNOME Project, 2026. [cit. 2026-03-29]. Dostupné z: https://gitlab.gnome.org/GNOME/zenity¨

[26] *GNU Make* [online]. Free Software Foundation, Inc., 2023. [cit. 2026-03-29]. Dostupné z: https://www.gnu.org/software/make/

[27] *GCC, the GNU Compiler Collection* [online]. Free Software Foundation, Inc., 2023. [cit. 2026-03-29]. Dostupné z: https://gcc.gnu.org/

[28] PRESTON-WERNER Tom, 2026. *TOML* [online]. 2026. [cit. 2026-03-29]. Dostupné z: https://toml.io/en/